TESTING NEURAL NETWORKS ON MOBILE DEVICES

Hung Manh Nguyen

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Department of Applied Mathematics
Study program: Informatics
Specialisation: Artificial intelligence
Supervisor: Ing. Jan Čech, Ph.D.
April 20, 2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Nguyen** Jméno: **Hung Manh** Osobní číslo: **515728**

Fakulta/ústav: **Fakulta informačních technologií**

Zadávající katedra/ústav: **Katedra aplikované matematiky**

Studijní program: **Informatika**

Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Testování neuronových sítí na mobilních zařízeních

Název bakalářské práce anglicky:

Testing Neural Networks on Mobile Devices

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Jan Čech, Ph.D. skupina vizuálního rozpoznávání FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **13.02.2026** Termín odevzdání bakalářské práce: _____

podpis vedoucí(ho) ústavu/katedry

podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Nguyen Hung Manh

Podpis studenta

© ČVUT v Praze, Design: ČVUT v Praze, VIC

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Nguyen** Jméno: **Hung Manh** Osobní číslo: **515728**

Fakulta/ústav: **Fakulta informačních technologií**

Zadávající katedra/ústav: **Katedra aplikované matematiky**

Studijní program: **Informatika**

Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Testování neuronových sítí na mobilních zařízeních

Název bakalářské práce anglicky:

Testing Neural Networks on Mobile Devices

Pokyny pro vypracování:

The goal of this thesis is to experimentally evaluate the feasibility of running modern neural networks for face processing on mobile devices (Android, iOS). The student will compare different architectures, frameworks, and device types in terms of speed, energy consumption, and accuracy. The work will focus primarily on real-time face detection and facial landmark estimation from video.

Tasks

1. Familiarize yourself with the options for running neural networks on mobile platforms.
2. Select suitable models for facial landmark detection and collect several test videos.
3. Implement a testing scenario on both Android and iOS.
4. Evaluate latency (ms/frame), throughput (fps), and optionally energy consumption. Compare results across several different devices.
5. Optionally, explore various optimization methods (e.g., quantization) and evaluate their impact on speed and accuracy.

References

[1] Li, Zhuojin, Marco Paolieri, and Leana Golubchik. "A benchmark for ML inference latency on mobile devices." Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking. 2024.
[2] Lee, Juhyun, et al. "On-device neural net inference with mobile GPUs." arXiv preprint arXiv:1907.01989 (2019).
[3] Wang, Siqi, Anuj Pathania, and Tulika Mitra. "Neural network inference on mobile SoCs." IEEE Design & Test 37.5 (2020): 50-57.
[4] Suchanek, Matej. "Efficient Implementation of Neural Networks for Real-Time Applications." Bachelor's thesis. Czech Technical University in Prague, 2020.

Seznam doporučené literatury:

 © ČVUT v Praze, Design: ČVUT v Praze, VIC

Czech Technical University in Prague
Faculty of Information Technology
© 2026 Hung Manh Nguyen. All rights reserved.
This thesis is school work as defined by Copyright Act of the Czech Republic. It has been submitted at Czech Technical University in Prague, Faculty of Information Technology. The thesis is protected by the Copyright Act and its use, with the exception of free statutory licenses and beyond the scope of the authorizations specified in the Declaration, requires the consent of the author.

Citation of this thesis: Nguyen Hung Manh. *Testing Neural Networks on Mobile Devices.* Bachelor's thesis. Czech Technical University in Prague, Faculty of Information Technology, 2026.

I would like to thank my supervisor Ing. Jan Čech, Ph.D. for his guidance, feedback, and support throughout this thesis.
I also thank my family and friends for their encouragement.

Declaration

I hereby declare that the presented thesis is my own work and that I have cited all sources of information in accordance with the Guideline for adhering to ethical principles when elaborating an academic final thesis.

I acknowledge that my thesis is subject to the rights and obligations stipulated by the Act No. 121/2000 Coll., the Copyright Act, as amended, in particular that the Czech Technical University in Prague has the right to conclude a license agreement on the utilization of this thesis as a school work under the provisions of Article 60 (1) of the Act.

I declare that I have used AI tools during the preparation and writing of my thesis. I have verified the generated content. I confirm that I am aware that I am fully responsible for the content of the thesis.

In Praze on April 20, 2026

Abstract

This thesis focuses on the theoretical foundations and benchmarking methodology for mobile face-processing neural networks on Android and iOS devices. It discusses mobile hardware constraints, efficient model design, quantization principles, and the ExecuTorch deployment workflow. The work defines a reproducible protocol for measuring end-to-end latency, throughput, and facial landmark accuracy on AFW, including data logging and validity controls for cross-platform evaluation.

Keywords mobile inference, face detection, facial landmarks, benchmarking, quantization, ExecuTorch, Core ML, XNNPACK

Abstrakt

Práce se věnuje teoretickým základům a metodice benchmarkingu neuronových sítí pro zpracování obličeje na mobilních zařízeních (Android, iOS). Popisuje omezení mobilního hardwaru, principy efektivních architektur a kvantizace, a detailně rozebírá deployment workflow frameworku ExecuTorch. Součástí je návrh reprodukovatelné metodiky pro měření end-to-end latence, propustnosti a přesnosti odhadu obličejových bodů na datasetu AFW.

Klíčová slova mobilní inference, detekce obličeje, obličejové body, benchmark, kvantizace, ExecuTorch, Core ML, XNNPACK

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem statement . . . 1
1.3 Research objectives . . . 2
1.4 Alignment with thesis assignment tasks . . . 2
1.5 Thesis structure . . . 2

2 Theoretical background **3**
2.1 On-device inference in mobile environments . . . 3
2.2 Hardware and system constraints . . . 3
2.2.1 CPU, GPU, and NPU roles . . . 3
2.2.2 Power and thermal behavior . . . 4
2.3 Efficient neural networks for mobile vision . . . 4
2.4 Model compression and quantization . . . 4
2.4.1 What PTQ does . . . 4
2.4.2 INT8 design choices that affect quality . . . 5
2.5 Face detection and landmark estimation . . . 5
2.5.1 Task definitions . . . 5
2.5.2 Representative models . . . 5
2.5.3 Datasets and annotation uncertainty . . . 6

3 ExecuTorch for mobile deployment **7**
3.1 Why ExecuTorch . . . 7
3.2 Conceptual architecture . . . 7
3.3 Export and lowering workflow . . . 7
3.4 Delegation and partitioning . . . 8
3.5 Quantization in ExecuTorch . . . 8
3.5.1 What INT8 PTQ means in this deployment pipeline . . . 8
3.5.2 Implemented PTQ workflow in the export scripts . . . 9
3.5.3 Model-specific safety workarounds used during quantized export . . . 9
3.5.4 Export-script parameters relevant to quantization . . . 10

4 Benchmark methodology **11**
4.1 Methodological goals . . . 11
4.2 Evaluated pipeline . . . 11

4.3 Model families and configuration space 11
4.4 iOS and Android implementation parity 12
4.5 Dataset and inputs 12
4.6 Metrics 12
4.6.1 Landmark inference latency 12
4.6.2 Throughput and batch scaling 13
4.6.3 Landmark accuracy 13
4.6.4 Optional energy metrics 13
4.7 Measurement protocol 13
4.8 Run artifacts and offline analysis 14
4.9 Threats to validity 14

5 Experimental results **15**
5.1 Result scope and source artifacts 15
5.2 BS=1 speed and latency overview (PC vs focus device) 15
5.3 Focus-device backend and quantization at BS=1 16
5.4 Batch scaling on focus device (BS=1,2,4,8) 18
5.5 Quantization deltas and cumulative error behavior 19
5.6 iPhone 14 charts, tables, and interpretation 19

6 Conclusion **25**

A Reproducibility **26**
A.1 Generate combined CSV and chart suite (used in thesis) 26
A.2 Generate summary tables (used in thesis) 26
A.3 Export INT8 XNNPACK landmark models (PTQ) 26

Bibliography **28**

Contents of the attachment **32**

List of Figures

5.1 BS=1 median FPS: PC vs focus device (from combined charts). 15
5.2 BS=1 median landmark-inference latency: PC vs focus device (from combined charts). 16
5.3 Focus device: BS=1 speed by backend and quantization. 16
5.4 Focus device: BS=1 NME accuracy by backend and quantization (lower is better). 17
5.5 Focus device batch scaling for PFLD-MNv2-1.0 (XNNPACK variants). 18
5.6 Focus device batch scaling for PipNet ResNet18 (XNNPACK variants). 18
5.7 Focus device: INT8 vs FP32 speed comparison at BS=1. 19
5.8 Focus device: INT8 vs FP32 NME difference at BS=1. 20
5.9 Focus device cumulative NME curves for BS=1 XNNPACK FP32/INT8 runs. 20
5.10 iPhone 14: BS=1 median FPS (device-focused chart pack). . . 21
5.11 iPhone 14: BS=1 median landmark-inference latency. 21
5.12 iPhone 14: BS=1 speed by backend and quantization. 21
5.13 iPhone 14: INT8 and FP32 speed comparison at BS=1. 22
5.14 iPhone 14 batch scaling for PFLD-MNv2-1.0. 23
5.15 iPhone 14 batch scaling for PipNet ResNet18. 23

List of Tables

2.1 Qualitative properties of models used in the benchmark matrix. 6

5.1 Focus-device BS=1 XNNPACK metrics (raw values). 17
5.2 Focus-device batch scaling in FPS (raw values). 19
5.3 Focus-device batch scaling in landmark-inference latency (raw values). 19
5.4 BS=1 PC INT8 and focus-device INT8 raw metrics. 20
5.5 iPhone 14 BS=1 throughput by backend (raw values). 20

5.6 iPhone 14 BS=1 end-to-end latency by backend (raw values). . 22
5.7 iPhone 14 batch scaling in FPS (raw values). 22
5.8 iPhone 14 batch scaling in landmark-inference latency (raw values). 23
5.9 BS=1 PC INT8 and iPhone 14 INT8 raw metrics. 23

List of abbreviations

AFW	Annotated Faces in the Wild (dataset)
AUC	Area Under the Curve
CPU	Central Processing Unit
Core ML	Apple on-device machine learning framework
E2E	End-to-end
FPS	Frames per second
INT8	8-bit integer quantization
IoU	Intersection over Union
NME	Normalized Mean Error
NNAPI	Android Neural Networks API
NPU	Neural Processing Unit
PFLD	Practical Facial Landmark Detector
XNNPACK	Optimized CPU backend used by ExecuTorch

Chapter 1
Introduction

This thesis addresses the engineering problem of deploying and evaluating modern face-processing neural networks directly on mobile devices. The work focuses on the theoretical foundations and experimental methodology required for trustworthy benchmarking of face detection and facial landmark estimation on Android and iOS using ExecuTorch and backend delegates.

1.1 Motivation

On-device inference has become a key requirement for many applications, because it improves privacy, reduces round-trip latency, and allows offline operation. At the same time, mobile deployment is constrained by strict limits on compute throughput, memory bandwidth, power, and thermal budget [1, 2]. In practice, this means that achieving good model accuracy is not sufficient; the full end-to-end system must also be efficient.

Recent community benchmarks confirm that measured latency strongly depends on hardware configuration, runtime backend, numerical precision, and measurement protocol [3, 2].

1.2 Problem statement

The core of this thesis is to evaluate mobile face-processing pipelines implemented with ExecuTorch. The benchmark must support equivalent iOS and Android applications, consistent measurement conditions, and validation of quality metrics.

The benchmarked workload is an end-to-end two-stage pipeline:

1. face detection in an input image,
2. facial landmark estimation on detected face crops.

1.3 Research objectives

Objectives of the thesis are:

- summarize practical options for running neural networks on Android and iOS;
- select suitable face-processing models (detector + landmark regressor) for mobile real-time use;
- implement equivalent benchmark scenarios on both platforms;
- evaluate latency (ms/frame), throughput (FPS), and landmark accuracy;
- analyze INT8 PTQ optimization impact on speed and quality.

Energy-related measurements are treated as optional/auxiliary in this thesis, depending on data availability and measurement reliability per device.

1.4 Alignment with thesis assignment tasks

The assignment tasks are covered as follows:

1. **Mobile NN deployment options:** Chapter 2 and Chapter 3 review mobile hardware/runtime constraints and ExecuTorch backend options.
2. **Model selection + test inputs:** Chapter 2 (model section) explains selected detector/landmark architectures; Chapter 4 defines the reproducible AFW-based input protocol used for quantitative comparison.
3. **Android/iOS testing scenario:** Chapter 4 documents feature parity and unified measurement schema for both mobile apps.
4. **Latency/FPS (and optional energy):** Chapter 4 defines metrics and logging; Chapter 5 reports measured cross-device results.
5. **Optimization impact (quantization):** Chapter 3 explains PTQ export workflow and Chapter 5 compares FP32 vs INT8 behavior.

1.5 Thesis structure

Chapter 2 introduces theoretical background: mobile inference constraints, efficient model design, quantization, and face-processing tasks. Chapter 3 explains ExecuTorch architecture and backend delegation for mobile deployment. Chapter 4 defines the benchmark methodology, metrics, protocol, and validity controls. Chapter 5 presents and interprets experimental results. The final chapter concludes the thesis.

In-text citations and bibliography records are formatted using the numeric method of ČSN ISO 690 via the `biblatex-iso690` style.

Chapter 2

Theoretical background

2.1 On-device inference in mobile environments

Mobile AI systems are typically deployed on heterogeneous Systems-on-Chip (SoCs), where CPU, GPU, and dedicated NPUs coexist [4, 1]. Compared to server inference, mobile workloads operate under tighter thermal and power limits and often share resources with other foreground tasks.

Latency depends on several things:

- model architecture (operator types, tensor shapes, parameter count),
- runtime backend (kernel library, operator fusion, threading policy),
- hardware mapping (CPU/GPU/NPU dispatch),

As a consequence, the same neural network can show substantially different performance across devices and software stacks [2, 3].

2.2 Hardware and system constraints

2.2.1 CPU, GPU, and NPU roles

CPUs provide broad operator coverage and predictable behavior, but may not offer the highest throughput for dense tensor computations. Mobile GPUs can improve throughput for selected workloads, while NPUs may deliver superior energy-efficiency when supported operators map well to accelerator kernels [4, 1]. In real deployments, unsupported subgraphs frequently fall back to CPU, and thus synchronization and memory transfer overhead become issues.

2.2.2 Power and thermal behavior

Peak performance is often not sustainable in mobile devices. Very long inference can trigger thermal throttling, reducing frequency and increasing latency. [2].

2.3 Efficient neural networks for mobile vision

The shift from CNNs deployed on servers to mobile-friendly architectures was enabled by operator-level efficiency ideas such as depthwise separable convolution, inverted residual blocks, and channel shuffle [5, 6, 7]. SqueezeNet demonstrated that competitive accuracy can be achieved with significantly fewer parameters [8].

For mobile face-processing, both detector and landmark models must be efficient.

2.4 Model compression and quantization

Compression and quantization reduce memory footprint and often improve runtime performance. Classical compression includes pruning and weight sharing [9]. In practical mobile deployment, integer quantization is especially important because INT8 kernels can be faster and more energy-efficient than FP32 on supported hardware [10].

2.4.1 What PTQ does

Post-training quantization (PTQ) converts an already trained FP32 model to a lower-precision representation, usually without retraining. In this thesis, the main target is INT8 execution. A real value x is represented by an integer q using scale s and zero-point z:

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s}\right) + z,\ q_{\min}, q_{\max}\right), \qquad \hat{x} = s\,(q - z). \tag{2.1}$$

For signed INT8, q is typically in $[-128, 127]$ (or $[-127, 127]$ in symmetric weight quantization). The reconstructed value $\hat{x}$ approximates x, so a small quantization error is introduced.

PTQ typically has three steps:

1. prepare the graph for quantization,
2. run calibration data through the model to estimate activation ranges,
3. convert operators to quantized kernels.

Compared to quantization-aware training (QAT), PTQ is much simpler to apply, but it is more sensitive to calibration quality and to model-specific outliers.

2.4.2 INT8 design choices that affect quality

Two design choices are central for this work:

- **Per-tensor vs per-channel weights:** per-channel assigns separate scales per output channel and usually preserves accuracy better, especially in convolution-heavy models.
- **Static vs dynamic activations:** static PTQ calibrates activation ranges offline; dynamic approaches estimate part of the range online. Static PTQ is often faster when backend support is strong.

INT8 often improves speed for two reasons: lower memory traffic (INT8 tensors are 4× smaller than FP32) and faster vectorized kernels. However, real end-to-end gains depend on operator coverage. If part of the graph falls back to non-quantized execution, conversion overhead can reduce the expected speedup [11].

2.5 Face detection and landmark estimation

2.5.1 Task definitions

Face detection finds faces with bounding boxes. Landmark estimation predicts keypoint coordinates (e.g., eye corners, nose tip, mouth corners), usually on a cropped face image. These tasks are strongly coupled: detection quality and crop geometry directly influence landmark quality.

2.5.2 Representative models

RetinaFace is a strong one-stage detector with high robustness under challenging conditions [12]. FaceBoxes is a lightweight detector explicitly designed for CPU real-time operation [13].

For landmarks, PFLD emphasizes compactness and speed [14]. MobileFaceNet provides a lightweight backbone originally developed for mobile face recognition and often reused for landmark regression [15]. PIPNet combines heatmap-style reasoning with low-resolution efficient heads for practical landmark localization [16].

In the benchmark matrix used in this thesis, the main detector+landmark combinations are based on RetinaFace-MNet0.25 or FaceBoxesV2 with MobileFaceNet, PFLD-MNv2-0.25, PFLD-MNv2-1.0, and PipNet-ResNet18 (plus PFLD-Blaze in selected runs).

Model	Stage	Relative cost	Main properties
RetinaFace-MNet0.25	Detector	Medium	Robust detection quality; higher detector cost than very light alternatives.
FaceBoxesV2	Detector	Low	Very fast detector; useful when throughput is prioritized.
MobileFaceNet	Landmark	Low	Lightweight baseline; stable and efficient for mobile CPUs.
PFLD-MNv2-0.25	Landmark	Low	Compact model with good speed and moderate landmark precision.
PFLD-MNv2-1.0	Landmark	Medium	Higher capacity than 0.25 variant; better potential quality at higher compute cost.
PipNet-ResNet18	Landmark	High	Stronger landmark localization capacity, but substantially heavier compute load.

Table 2.1 Qualitative properties of models used in the benchmark matrix.

2.5.3 Datasets and annotation uncertainty

In this thesis, AFW annotations are used for wild-condition landmark evaluation, originating from the face detection and pose estimation work of Zhu and Ramanan [17].

Chapter 3

ExecuTorch for mobile deployment

3.1 Why ExecuTorch

ExecuTorch is a PyTorch edge deployment stack designed for efficient on-device inference across mobile and embedded targets [18, 19]. Its central design principle is *ahead-of-time* (AOT) preparation: expensive transformation and planning steps are performed before deployment, so runtime execution remains lightweight.

3.2 Conceptual architecture

ExecuTorch organizes model deployment into three phases [18, 19]:

1. export a PyTorch model to graph form,
2. lower and optimize the graph for edge execution,
3. execute the serialized program with a small C++ runtime.

3.3 Export and lowering workflow

The practical workflow consists of:

1. model export using `torch.export`,
2. backend specific lowering via `to_edge_transform_and_lower`,
3. creation of a serialized `.pte` artifact.

This process supports dynamic shape constraints and optional separation of program and tensor data into `.pte` and `.ptd` files [20, 21, 22, 23].

3.4 Delegation and partitioning

ExecuTorch uses delegates to offload supported subgraphs to backend specific implementations [24]. A partitioner tags lowerable graph regions; unsupported regions remain in portable execution. This mixed execution model improves portability but can create overhead.

For this thesis, two backends are central:

- **XNNPACK**: optimized CPU backend with broad mobile support, including quantized paths [25, 26];
- **Core ML delegate**: Apple backend path that can dispatch to CPU, GPU, or ANE depending on model and platform capabilities [27, 28].

3.5 Quantization in ExecuTorch

ExecuTorch implements quantization through backend specific configurations, commonly using PT2E/torchao-based preparation and conversion [29, 30]. In this project, INT8 exports are produced for XNNPACK by `models/export_landmark_models_xnnpac` which applies symmetric PTQ.

3.5.1 What INT8 PTQ means in this deployment pipeline

In practical terms, the exported graph replaces supported FP32 operators with INT8 kernels (mainly convolution and linear-family operations in XNNPACK). The internal arithmetic is typically INT8×INT8 with INT32 accumulation, followed by requantization to the next expected tensor format.

This usually helps mobile inference because:

- weights and (for static paths) activations use less memory,
- cache locality improves,
- backend kernels can use optimized vector instructions.

However, measured gains depend on full-graph compatibility. If unsupported operators force fallback segments, extra quantize/dequantize boundaries appear and reduce net speed benefit. This explains why model-wise INT8 speedups in the results chapter are not uniform.

For landmark regression, INT8 error sensitivity is model-dependent: models with stronger redundancy can absorb quantization noise better, while compact or outlier-sensitive layers may show visible NME degradation if calibration is not representative.

3.5.2 Implemented PTQ workflow in the export scripts

For each model and selected quantization method, the script runs:

1. build XNNPACK quantizer configuration,
2. prepare quantized graph with `prepare_pt2e`,
3. run calibration forward passes,
4. convert to quantized graph with `convert_pt2e`,
5. lower to ExecuTorch with XNNPACK partitioner and serialize `.pte`.

Two INT8 methods are implemented:

- `pc`: per-channel weight quantization (file suffix `_xnnpack_int8`),
- `pt`: per-tensor weight quantization (file suffix `_xnnpack_int8_pt`).

The quantizer setup is created by `get_symmetric_quantization_config(is_per_channel, is_dynamic)`. Hence the key internal PTQ parameters are:

- `is_per_channel` $\in \{\texttt{True}, \texttt{False}\}$,
- `is_dynamic` $\in \{\texttt{True}, \texttt{False}\}$.

Static INT8 export uses a fixed sample input tensor and calibrates for S iterations, where

$$S = \texttt{XNNPACK_INT8_CALIBRATION_SAMPLES} \tag{3.1}$$

(default $S = 32$).

Dynamic-batch INT8 export constrains only the batch dimension using `torch.export.Dim("batch", min, max)` and calibrates representative batch sizes from $\{b_{\min}, b_{\text{mid}}, b_{\max}\}$. In the script,

$$b_{\text{mid}} = \begin{cases} b_{\min}, & b_{\min} > 1, \\ \min(2, b_{\max}), & b_{\min} = 1. \end{cases} \tag{3.2}$$

3.5.3 Model-specific safety workarounds used during quantized export

The implemented export pipeline contains explicit safeguards discovered during practical deployment testing:

- **MobileFaceNet output stabilization:** fusion of `Linear+BatchNorm1d` in the output head and export as a single landmark tensor output;

- **MobileFaceNet XNNPACK stability:** exclusion of `PreluConfig` from XNNPACK delegation to avoid unstable runtime behavior;
- **MobileFaceNet static INT8 workaround:** static-shape export may reuse a dynamic-activation PTQ graph (controlled by `MOBILEFACENET_STATIC_INT8_DYNAMIC_ACT_`
- **Method filtering:** per-tensor INT8 is not used for MobileFaceNet in this export pipeline due to known instability.

3.5.4 Export-script parameters relevant to quantization

The quantized export process is controlled through environment parameters:

- `EXPORT_BATCH_SIZES`: static batch sizes to export (default in wrapper script: `1`);
- `EXPORT_DYNAMIC_BATCH`: enable dynamic-batch exports (default: `1`);
- `EXPORT_DYNAMIC_MIN_BATCH`: lower dynamic batch bound (default: `1`);
- `EXPORT_DYNAMIC_MAX_BATCH`: upper dynamic batch bound (default: `128`);
- `XNNPACK_INT8_CALIBRATION_SAMPLES`: number of static calibration iterations (default: `32`);
- `XNNPACK_INT8_METHODS`: selected INT8 methods (default: `pc,pt`);
- `MOBILEFACENET_STATIC_INT8_DYNAMIC_ACT_WORKAROUND`: enable MobileFaceNet static INT8 workaround (default: enabled);
- `PYTHON_BIN`: interpreter used by wrapper shell script.

For completeness, the same export ecosystem also accepts backend-target parameters used by non-quantized scripts (`QNN_SOC_MODEL`, `MTK_PLATFORM_CONFIG`, `SAMSUNG_CHIPSET`, `EXPORT_APPLE_BACKENDS`), which influence which delegate variants are generated but do not change INT8 calibration rules.

Chapter 4

Benchmark methodology

4.1 Methodological goals

The benchmark design in this thesis follows two goals:

1. produce comparable latency/throughput/accuracy measurements across iOS and Android,
2. preserve raw artifacts needed for analysis of the results.

4.2 Evaluated pipeline

Each benchmark run executes an end-to-end pipeline:

1. detector preprocessing,
2. detector inference,
3. detector postprocessing (box decoding and filtering),
4. face crop generation,
5. landmark model preprocessing,
6. landmark inference,
7. landmark postprocessing.

4.3 Model families and configuration space

The methodology supports multiple detector/landmark pairs and backend configurations. Typical factors include:

- detector choice (e.g., RetinaFace vs FaceBoxes),

- landmark architecture (e.g., PFLD, MobileFaceNet, PIPNet),
- precision variant (FP32 and INT8),
- backend delegate (XNNPACK, Core ML and portable),
- resolved batch size.

A complete run record stores these factors explicitly, so later analysis does not depend on file naming conventions alone.

4.4 iOS and Android implementation parity

Both iOS and Android benchmark applications are designed to be feature-equivalent at the pipeline level. Equivalent behavior includes:

- identical preprocessing parameters,
- identical postprocessing logic and thresholds,
- consistent metric collection and export schema,
- consistent warm-up and measured iterations.

This principle follows the project guideline that iOS and Android applications should remain equivalent in features and purpose.

4.5 Dataset and inputs

AFW-based landmark annotations are used for accuracy-oriented offline recomputation [17].

4.6 Metrics

4.6.1 Landmark inference latency

Although each run executes the full detector+landmark pipeline, the thesis comparison charts are organized by the *landmark* configuration (model, backend, quantization, and resolved batch size). The primary timing metric is therefore landmark *inference* latency only:

$$\ell_i^{\text{lm-inf}} = t_{\text{lm-inf},i}^{\text{end}} - t_{\text{lm-inf},i}^{\text{start}}. \tag{4.1}$$

For each run, this metric is extracted from per-iteration `landmarkInferenceMs` samples and summarized in milliseconds by average, $p50$, and $p95$. For charting, values are aggregated per configuration group using the median across repeated runs:

$$\tilde{m}_g = \text{median}\{m_r \mid r \in g\}. \tag{4.2}$$

4.6.2 Throughput and batch scaling

Run throughput follows the exported timing summary:

$$\mathrm{FPS} = \frac{N_{\mathrm{images}}}{T_{\mathrm{run}}}. \tag{4.3}$$

BS=1 charts compare median FPS by landmark configuration. Batch-scaling charts plot median FPS as a function of resolved landmark batch size ($b \in \{1, 2, 4, 8\}$ in the curated thesis set). Runs marked with runtime fallback to BS=1 are excluded from scaling comparisons.

4.6.3 Landmark accuracy

Accuracy is recomputed offline from exported per-image predicted landmarks and AFW ground truth annotations. For N evaluated images and K landmarks, inter-ocular NME is

$$\mathrm{NME}_{\mathrm{iod}} = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{K} \sum_{k=1}^{K} \frac{\|\hat{\mathbf{p}}_{n,k} - \mathbf{p}_{n,k}\|_2}{d_n^{\mathrm{iod}}}, \tag{4.4}$$

where d_n^{iod} is the inter-ocular distance for image n.

The analysis also keeps FR@0.08, FR@0.1 and AUC@0.1 as supplementary accuracy statistics, while the main thesis charts use median $\mathrm{NME}_{\mathrm{iod}}$ (lower is better). By analysis rule, accuracy plots use iOS rows with valid recomputation payloads (non-zero evaluated images).

4.6.4 Optional energy metrics

If platform tooling allows reliable collection, energy-related signals can be logged as supplementary indicators. In this thesis, landmark-inference latency, throughput, and landmark accuracy remain primary.

4.7 Measurement protocol

To reduce noise and measurement bias, each benchmark condition follows a fixed protocol:

- dedicated warm-up iterations before timed measurement,
- fixed number of measured iterations,
- stable device state (charging policy, background app control, thermal preconditioning),
- separate logging for initialization, inference, and optional postprocessing stages.

Runs that violate schema, contain missing key payloads, or fail consistency checks are flagged for exclusion from aggregate statistics.

4.8 Run artifacts and offline analysis

Every run exports a machine-readable summary plus raw samples. Offline analysis scripts recompute aggregate statistics and, when possible, recompute accuracy from per-image predictions instead of trusting prefilled summary fields. This post-hoc recomputation improves robustness against stale metadata and partial export errors.

4.9 Threats to validity

Main threats and mitigations include:

- **Thermal drift**: controlled warm-up and repeated runs;
- **Backend asymmetry**: shared-configuration comparisons for cross-device claims;
- **Unsupported operators**: explicit reporting of delegate coverage and fallback paths;

Chapter 5

Experimental results

5.1 Result scope and source artifacts

This chapter uses only generated artifacts from `benchmarks/results/combined`. Figures are loaded directly from `../benchmarks/results/combined/charts/*` and all numeric tables are based on CSV files in `benchmarks/results/combined/tables/*`.

The main simplified dashboard is configured around one focus mobile device (Xiaomi Redmi Note 8 Pro) with PC reference rows. In addition, per-device chart packs generated in `benchmarks/results/combined/charts/device_iphone_14` are used below to include iPhone 14 specific analysis and tables.

5.2 BS=1 speed and latency overview (PC vs focus device)

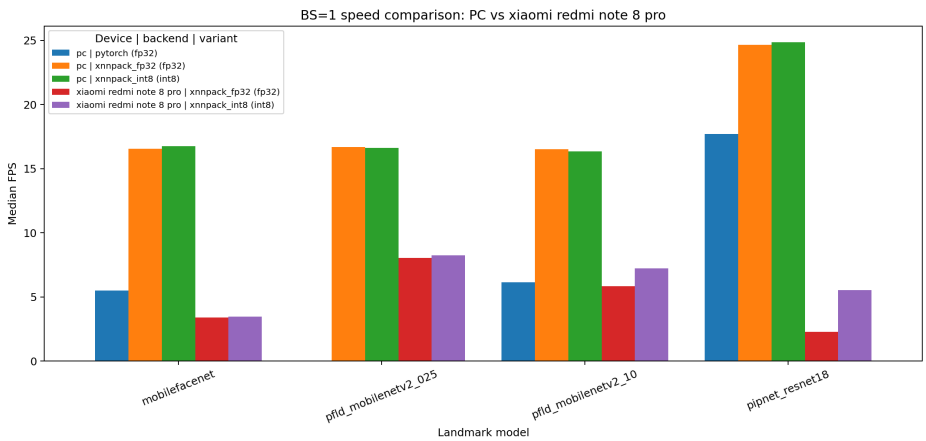

Figure 5.1 BS=1 median FPS: PC vs focus device (from combined charts).

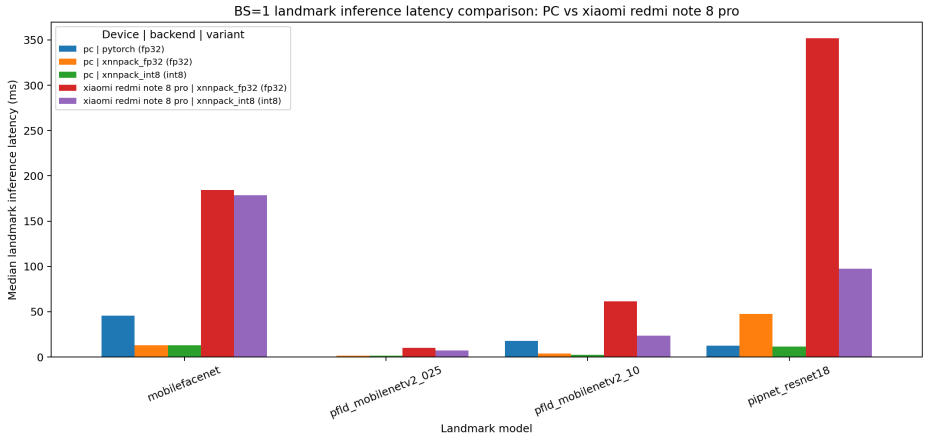

Figure 5.2 BS=1 median landmark-inference latency: PC vs focus device (from combined charts).

Figures 5.1 and 5.2 show the expected gap between desktop and mobile execution at BS=1. The PC reference is consistently faster and has lower landmark-inference latency, while the focus phone remains the practical on-device target for deployment-oriented testing.

5.3 Focus-device backend and quantization at BS=1

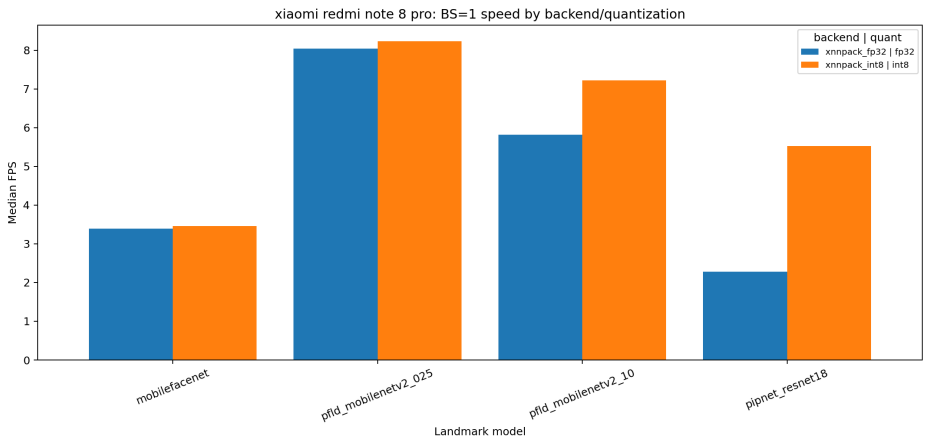

Figure 5.3 Focus device: BS=1 speed by backend and quantization.

All numeric table values in this chapter are taken from `benchmarks/results/combined/tables`, mainly `simple_bs1_pc_vs_focus_device.csv`, `simple_batch_scaling_pc_vs_focus_device.csv`, and `simple_pc_int8_vs_xiaomi_redmi_note_8_pro_xnn_speedup_bs1.csv`.

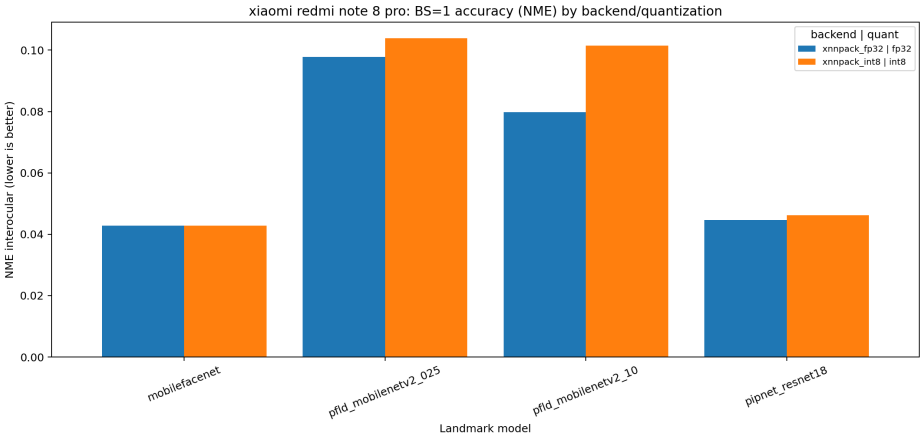

Figure 5.4 Focus device: BS=1 NME accuracy by backend and quantization (lower is better).

Model	Quant	FPS	LM inf. [ms]	E2E [ms]	NME
MobileFaceNet	FP32	3.39	184.33	277.18	0.0428
MobileFaceNet	INT8	3.46	178.39	272.20	0.0428
PFLD-MNv2-0.25	FP32	8.04	9.99	107.34	0.0978
PFLD-MNv2-0.25	INT8	8.24	7.10	104.81	0.1039
PFLD-MNv2-1.0	FP32	5.82	61.54	155.65	0.0798
PFLD-MNv2-1.0	INT8	7.22	23.78	121.56	0.1014
PipNet-R18	FP32	2.28	351.97	421.52	0.0446
PipNet-R18	INT8	5.53	97.46	164.59	0.0463

Table 5.1 Focus-device BS=1 XNNPACK metrics (raw values).

Table 5.1 confirms what is visible in Figures 5.3 and 5.4: INT8 improves runtime strongly for heavier models (especially PipNet ResNet18 and PFLD-MNv2-1.0), while accuracy impact is model-dependent. For MobileFaceNet, accuracy is almost unchanged; for PFLD-MNv2-1.0, NME increases.

5.4 Batch scaling on focus device (BS=1,2,4,8)

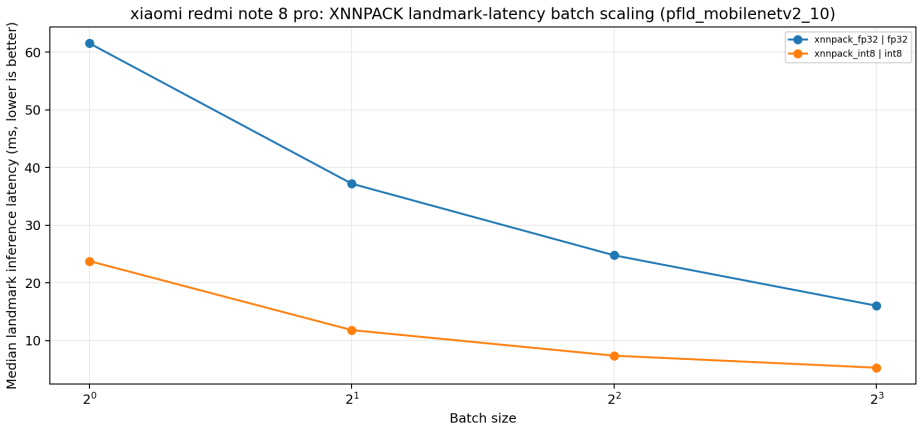

Figure 5.5 Focus device batch scaling for PFLD-MNv2-1.0 (XNNPACK variants).

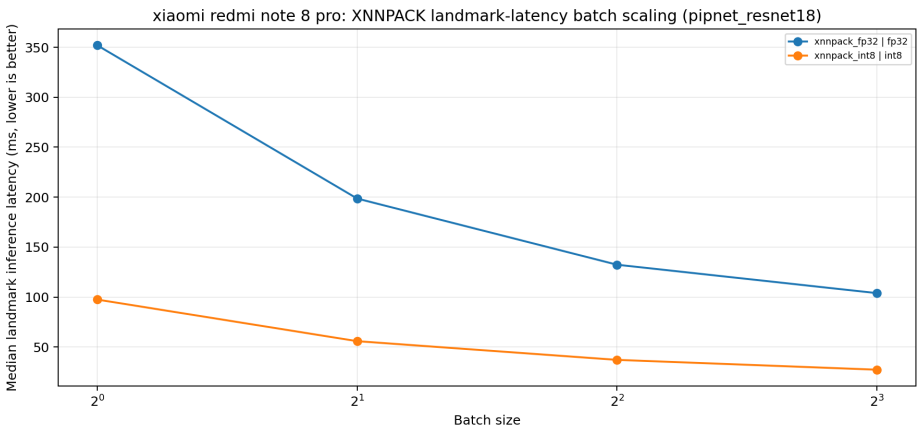

Figure 5.6 Focus device batch scaling for PipNet ResNet18 (XNNPACK variants).

The batch-scaling tables provide two complementary views. FPS stays relatively stable across larger batch sizes, while per-image landmark latency drops substantially. This indicates that batching makes landmark forward pass more efficient, but full pipeline throughput is constrained by additional stages (detector and surrounding pipeline steps).

Model	Quant	FPS@1	FPS@2	FPS@4	FPS@8
PFLD-MNv2-1.0	FP32	5.82	5.53	5.44	5.97
PFLD-MNv2-1.0	INT8	7.22	7.50	7.49	7.66
PipNet-R18	FP32	2.28	2.22	2.31	2.27
PipNet-R18	INT8	5.53	5.34	5.52	5.66

Table 5.2 Focus-device batch scaling in FPS (raw values).

Model	Quant	LM ms@1	LM ms@2	LM ms@4	LM ms@8
PFLD-MNv2-1.0	FP32	61.54	37.19	24.78	16.03
PFLD-MNv2-1.0	INT8	23.78	11.81	7.37	5.29
PipNet-R18	FP32	351.97	198.58	132.34	103.89
PipNet-R18	INT8	97.46	55.81	37.10	27.26

Table 5.3 Focus-device batch scaling in landmark-inference latency (raw values).

5.5 Quantization deltas and cumulative error behavior

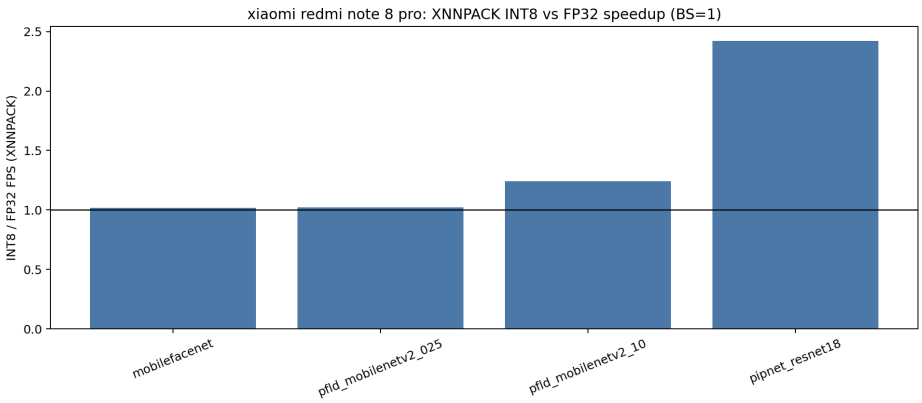

Figure 5.7 Focus device: INT8 vs FP32 speed comparison at BS=1.

Table 5.4 is intentionally numeric-only (no ratio columns). It gives direct deployment context: for every model, the same INT8 configuration is faster on PC, while the mobile path keeps practical on-device latency and accuracy characteristics for real usage.

5.6 iPhone 14 charts, tables, and interpretation

Tables 5.5 and 5.6 explain the main backend trend on iPhone 14. Core ML is consistently fastest because it can use highly optimized Apple kernels and delegate scheduling across Apple hardware. XNNPACK is slower than Core

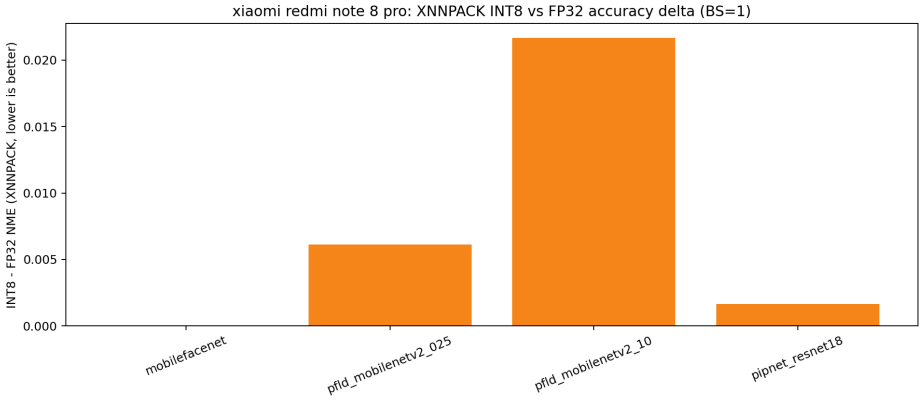

Figure 5.8 Focus device: INT8 vs FP32 NME difference at BS=1.

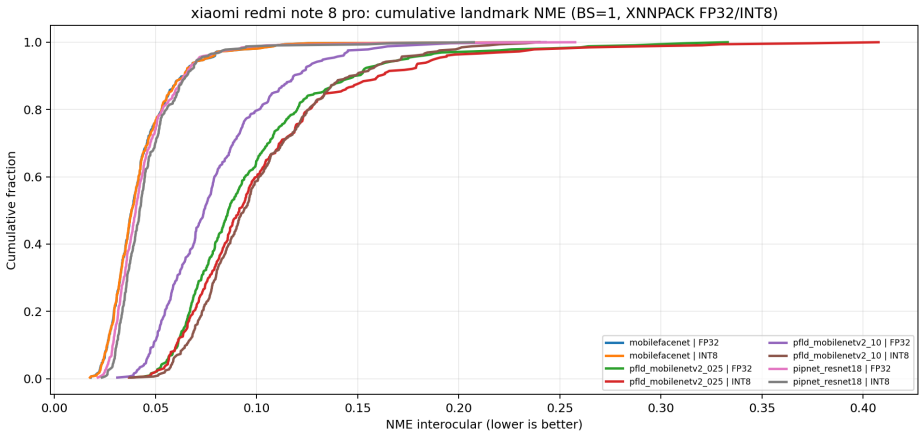

Figure 5.9 Focus device cumulative NME curves for BS=1 XNNPACK FP32/INT8 runs.

Model	PC INT8 FPS	Focus INT8 FPS	PC E2E [ms]	Focus E2E [ms]
MobileFaceNet	16.75	3.46	59.64	272.20
PFLD-Blaze	16.44	7.96	60.68	108.49
PFLD-MNv2-0.25	16.63	8.24	59.92	104.81
PFLD-MNv2-1.0	16.35	7.22	61.01	121.56
PipNet-R18	24.86	5.53	40.19	164.59

Table 5.4 BS=1 PC INT8 and focus-device INT8 raw metrics.

Model	Core ML FPS	XNN FP32 FPS	XNN INT8 FPS	Portable FPS
MobileFaceNet	8.45	5.14	5.38	0.30
PFLD-MNv2-0.25	8.63	6.59	6.76	0.50
PFLD-MNv2-1.0	8.51	5.78	6.34	0.25
PipNet-R18	7.08	2.81	4.94	0.07

Table 5.5 iPhone 14 BS=1 throughput by backend (raw values).

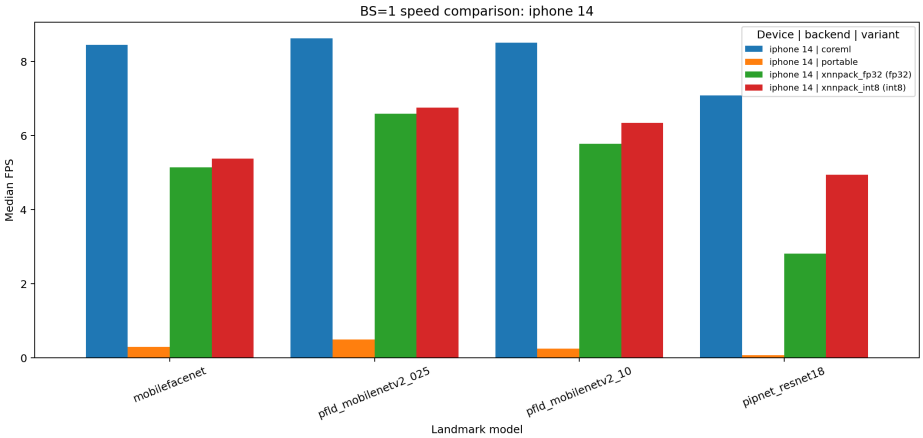

Figure 5.10 iPhone 14: BS=1 median FPS (device-focused chart pack).

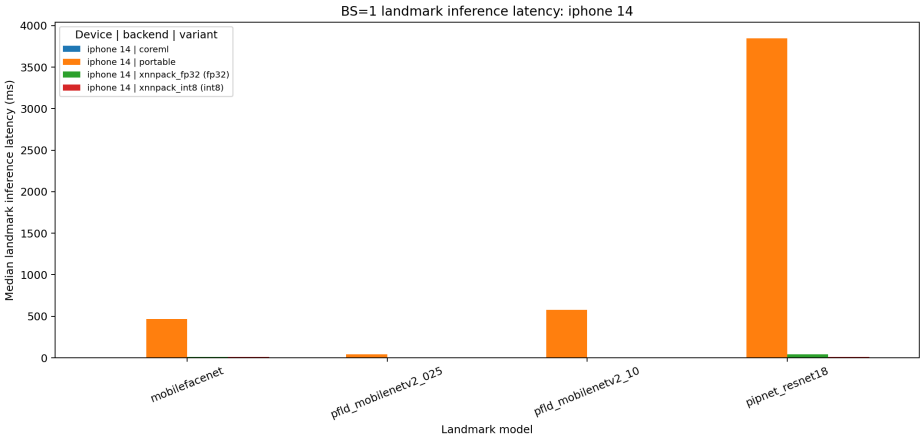

Figure 5.11 iPhone 14: BS=1 median landmark-inference latency.

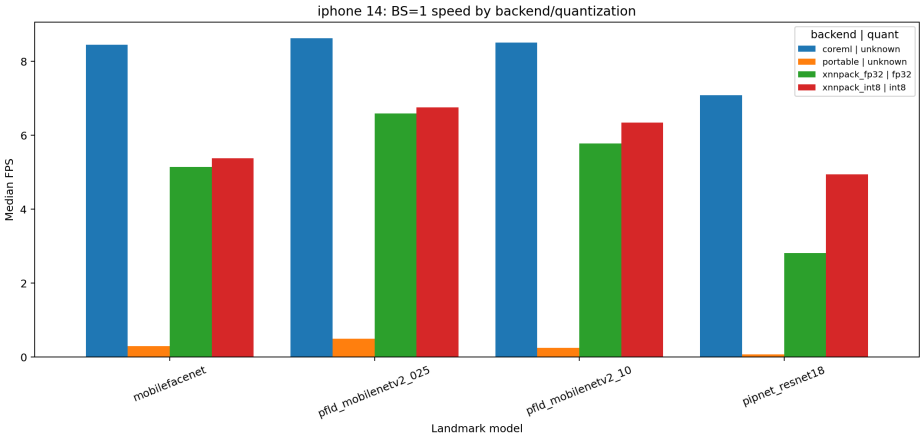

Figure 5.12 iPhone 14: BS=1 speed by backend and quantization.

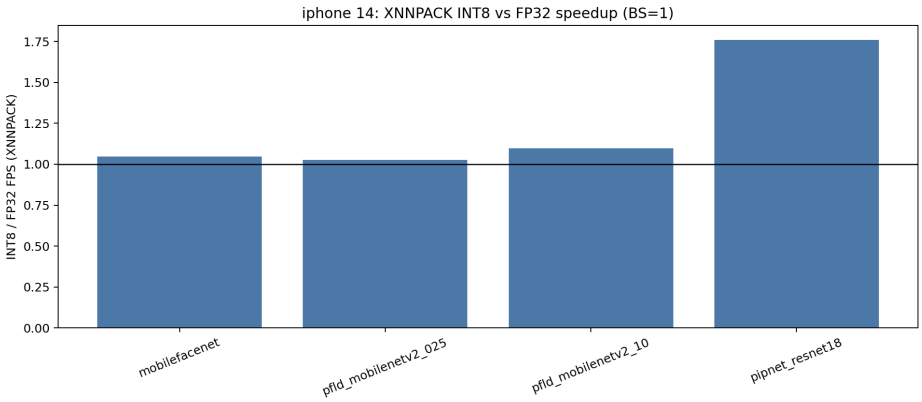

Figure 5.13 iPhone 14: INT8 and FP32 speed comparison at BS=1.

Model	Core ML E2E [ms]	XNN FP32 E2E [ms]	XNN INT8 E2E [ms]	Portable E2E [ms]
MobileFaceNet	107.35	177.74	169.33	3374.61
PFLD-MNv2-0.25	107.89	135.38	135.90	1999.97
PFLD-MNv2-1.0	109.89	160.36	141.19	3962.29
PipNet-R18	133.35	338.35	186.01	13698.42

Table 5.6 iPhone 14 BS=1 end-to-end latency by backend (raw values).

ML but remains practical. Portable execution is by far the slowest because it runs generic kernels without strong backend-specific acceleration.

INT8 gains depend on model complexity. For smaller models (MobileFaceNet, PFLD-MNv2-0.25), FPS gains are modest because detector and fixed pipeline costs already consume a large share of total runtime. For heavier models (PFLD-MNv2-1.0 and especially PipNet-R18), INT8 reduces compute and memory traffic enough to produce much larger end-to-end improvements.

Model	Quant	FPS@1	FPS@2	FPS@4	FPS@8
PFLD-MNv2-1.0	FP32	5.78	5.83	5.88	5.91
PFLD-MNv2-1.0	INT8	6.34	6.24	6.37	6.51
PipNet-R18	FP32	2.81	2.85	2.85	2.93
PipNet-R18	INT8	4.94	5.02	4.96	5.00

Table 5.7 iPhone 14 batch scaling in FPS (raw values).

The iPhone 14 batch-scaling pattern matches the focus-device behavior: FPS is relatively stable as batch increases, while per-image landmark latency drops strongly. The reason is that batching improves arithmetic efficiency inside the landmark forward pass, but detector, preprocessing, and postprocessing still limit overall pipeline FPS.

Table 5.9 gives direct context for deployment planning: PC remains faster for all tested models due higher compute throughput and memory bandwidth,

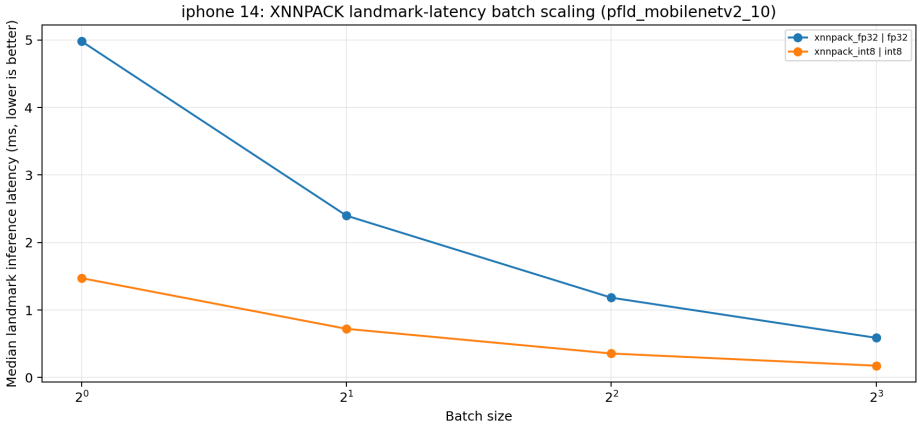

Figure 5.14 iPhone 14 batch scaling for PFLD-MNv2-1.0.

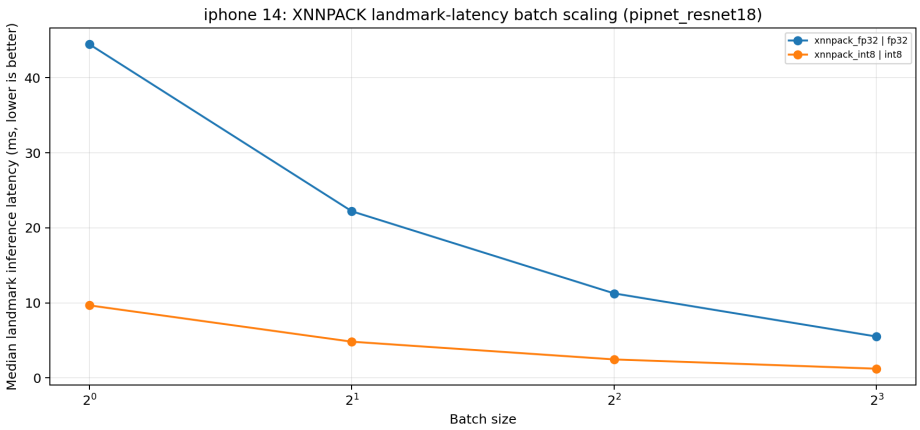

Figure 5.15 iPhone 14 batch scaling for PipNet ResNet18.

Model	Quant	LM ms@1	LM ms@2	LM ms@4	LM ms@8
PFLD-MNv2-1.0	FP32	4.98	2.40	1.18	0.59
PFLD-MNv2-1.0	INT8	1.47	0.72	0.36	0.17
PipNet-R18	FP32	44.45	22.20	11.26	5.51
PipNet-R18	INT8	9.66	4.82	2.46	1.22

Table 5.8 iPhone 14 batch scaling in landmark-inference latency (raw values).

Model	PC INT8 FPS	iPhone INT8 FPS	PC E2E [ms]	iPhone E2E [ms]
MobileFaceNet	16.75	5.38	59.64	169.33
PFLD-MNv2-0.25	16.63	6.76	59.92	135.90
PFLD-MNv2-1.0	16.35	6.34	61.01	141.19
PipNet-R18	24.86	4.94	40.19	186.01

Table 5.9 BS=1 PC INT8 and iPhone 14 INT8 raw metrics.

while iPhone 14 still provides stable on-device performance in the practical FPS range.

For accuracy, the current combined export does not include valid recomputed NME rows for iPhone 14 in this filtered dataset, so iPhone-specific accuracy charts are not generated here. This is a data-availability limitation, not a runtime measurement issue.

Chapter 6
Conclusion

This thesis presented a full benchmark workflow for mobile face-processing models with ExecuTorch, from theoretical background and export details to cross-platform runtime evaluation.

With respect to the assignment goals, the thesis delivered:

- implementation and comparison of mobile inference paths on both iOS and Android,
- evaluation of real-time-oriented metrics (ms/frame and FPS),
- multi-device comparison with reproducible logging and offline analysis,
- quantization study (INT8 PTQ) with speed/accuracy trade-off analysis,
- optional energy-related telemetry as auxiliary data where available.

Using only generated artifacts in `benchmarks/results/combined`, the results show four main points:

- backend and quantization selection strongly influence runtime;
- INT8 can provide major latency and FPS gains for heavier landmark models, but accuracy impact depends on the model;
- increasing batch size improves per-image landmark-inference latency, while end-to-end FPS may change only modestly;
- PC reference runs are useful for capacity comparison, but deployment decisions must be made from mobile-device measurements.

Overall, the implemented methodology and artifact pipeline provide a reproducible base for future iterations, including model replacement, backend retuning, and extension to more mobile devices.

Appendix A
Reproducibility

This appendix lists the commands used to regenerate the CSV data, charts, and tables referenced in the thesis.

A.1 Generate combined CSV and chart suite (used in thesis)

From the repository root:

```
./.venv/bin/python benchmarks/analysis/plot_runs_dashboard.py \
  --mobile-root benchmarks/results/mobile \
  --pc-root benchmarks/results/pc/runs \
  --focus-device "xiaomi redmi note 8 pro" \
  --max-resolved-batch 8 \
  --output-csv benchmarks/results/combined/combined_runs_from_json.csv \
  --output-dir benchmarks/results/combined/charts
```

A.2 Generate summary tables (used in thesis)

```
./.venv/bin/python benchmarks/analysis/summarize_benchmarks.py \
  --input benchmarks/results/combined/combined_runs_from_json.csv \
  --output-dir benchmarks/results/combined/tables \
  --focus-device "xiaomi redmi note 8 pro"
```

A.3 Export INT8 XNNPACK landmark models (PTQ)

```
EXPORT_BATCH_SIZES=1 \
EXPORT_DYNAMIC_BATCH=1 \
EXPORT_DYNAMIC_MIN_BATCH=1 \
EXPORT_DYNAMIC_MAX_BATCH=128 \
```

```
XNNPACK_INT8_CALIBRATION_SAMPLES=32 \
XNNPACK_INT8_METHODS=pc,pt \
./models/export_landmark_models_xnnpack_quantized.sh
```

Optional MobileFaceNet stability switch (enabled by default):

```
MOBILEFACENET_STATIC_INT8_DYNAMIC_ACT_WORKAROUND=1
```

Bibliography

1. WANG, Siqi; PATHANIA, Anuj; MITRA, Tulika. Neural Network Inference on Mobile SoCs. *IEEE Design & Test*. 2020, vol. 37, no. 5, pp. 50–57. Available from DOI: 10.1109/MDAT.2020.2968258.
2. LI, Zhuojin; PAOLIERI, Marco; GOLUBCHIK, Leana. A Benchmark for ML Inference Latency on Mobile Devices. In: *Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking (EdgeSys '24)*. 2024. Available from DOI: 10.1145/3642968.3654818.
3. REDDI, Vijay Janapa; KANTER, David; MATTSON, Peter; DUKE, Jared; NGUYEN, Thai; CHUKKA, Ramesh; SHIRING, Ken; TAN, Koan-Sin; CHARLEBOIS, Mark; CHOU, William, et al. MLPerf Mobile Inference Benchmark: An Industry-Standard Open-Source Machine Learning Benchmark for On-Device AI. *Proceedings of Machine Learning and Systems* [online]. 2022, vol. 4, pp. 352–369 [visited on 2026-04-19]. Available from: https://proceedings.mlsys.org/paper_files/paper/2022/hash/a2b2702ea7e682c5ea2c20e8f71efb0c-Abstract.html.
4. LEE, Juhyun; CHIRKOV, Nikolay; IGNASHEVA, Ekaterina; PISARCHYK, Yury; SHIEH, Mogan; RICCARDI, Fabio; SAROKIN, Raman; KULIK, Andrei; GRUNDMANN, Matthias. On-Device Neural Net Inference with Mobile GPUs. *arXiv preprint arXiv:1907.01989*. 2019. Available from DOI: 10.48550/arXiv.1907.01989.
5. HOWARD, Andrew G.; ZHU, Menglong; CHEN, Bo; KALENICHENKO, Dmitry; WANG, Weijun; WEYAND, Tobias; ANDREETTO, Marco; ADAM, Hartwig. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*. 2017. Available from DOI: 10.48550/arXiv.1704.04861.
6. SANDLER, Mark; HOWARD, Andrew; ZHU, Menglong; ZHMOGINOV, Andrey; CHEN, Liang-Chieh. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In: *Proceedings of the IEEE/CVF Conference on Com-*

puter Vision and Pattern Recognition (CVPR). 2018, pp. 4510–4520. Available from DOI: 10.1109/CVPR.2018.00474.

7. ZHANG, Xiangyu; ZHOU, Xinyu; LIN, Mengxiao; SUN, Jian. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 6848–6856. Available from DOI: 10.1109/CVPR.2018.00716.
8. IANDOLA, Forrest N.; MOSKEWICZ, Matthew W.; ASHRAF, Khalid; HAN, Song; DALLY, William J.; KEUTZER, Kurt. SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <1MB Model Size. *arXiv preprint arXiv:1602.07360*. 2016. Available from DOI: 10.48550/arXiv.1602.07360.
9. HAN, Song; MAO, Huizi; DALLY, William J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *arXiv preprint arXiv:1510.00149*. 2016. Available from DOI: 10.48550/arXiv.1510.00149.
10. JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2704–2713. Available from DOI: 10.1109/CVPR.2018.00286.
11. KRISHNAMOORTHI, Raghuraman. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper. *arXiv preprint arXiv:1806.08342*. 2018. Available from DOI: 10.48550/arXiv.1806.08342.
12. DENG, Jiankang; GUO, Jia; VERVERAS, Evangelos; KOTSIA, Irene; ZAFEIRIOU, Stefanos. RetinaFace: Single-Stage Dense Face Localisation in the Wild. *arXiv preprint arXiv:1905.00641*. 2020. Available from DOI: 10.48550/arXiv.1905.00641.
13. ZHANG, Shifeng; ZHU, Xiangyu; LEI, Zhen; SHI, Hailin; WANG, Xiaobo; LI, Stan Z. FaceBoxes: A CPU Real-Time Face Detector with High Accuracy. In: *2017 IEEE International Joint Conference on Biometrics (IJCB)*. 2017, pp. 1–9. Available from DOI: 10.1109/BTAS.2017.8272675.
14. GUO, Xiaojie; LI, Siyuan; YU, Jinke; ZHANG, Jiawan; MA, Jiayi; MA, Lin; LIU, Wei; LING, Haibin. PFLD: A Practical Facial Landmark Detector. *arXiv preprint arXiv:1902.10859*. 2019. Available from DOI: 10.48550/arXiv.1902.10859.

15. CHEN, Sheng; LIU, Yang; GAO, Xiang; HAN, Zhen. MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices. In: *Biometric Recognition (CCBR 2018)*. Springer, 2018, vol. 10996, pp. 428–438. Lecture Notes in Computer Science. Available from DOI: 10.1007/978-3-319-97909-0_46.

16. JIN, Haibo; LIAO, Shengcai; SHAO, Ling. Pixel-in-Pixel Net: Towards Efficient Facial Landmark Detection in the Wild. *International Journal of Computer Vision*. 2021, vol. 129, no. 12, pp. 3174–3194. Available from DOI: 10.1007/s11263-021-01521-4.

17. ZHU, Xiangxin; RAMANAN, Deva. Face Detection, Pose Estimation, and Landmark Localization in the Wild. In: *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 2879–2886. Available from DOI: 10.1109/CVPR.2012.6248014.

18. PYTORCH FOUNDATION. *How ExecuTorch Works* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/intro-how-it-works.html.

19. PYTORCH FOUNDATION. *Architecture and Components — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/getting-started-architecture.html.

20. PYTORCH FOUNDATION. *Model Export and Lowering — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/using-executorch-export.html.

21. PYTORCH FOUNDATION. *.pte file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/pte-file-format.html.

22. PYTORCH FOUNDATION. *.ptd file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/ptd-file-format.html.

23. PYTORCH FOUNDATION. *torch.export User Guide* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/export.html.

24. PYTORCH FOUNDATION. *Understanding Backends and Delegates — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/compiler-delegate-and-partitioner.html.

25. PYTORCH FOUNDATION. *XNNPACK Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/xnnpack/xnnpack-overview.html.

26. GOOGLE. *XNNPACK: High-efficiency Neural Network Operators* [online]. 2026. [visited on 2026-04-14]. Available from: https://github.com/google/XNNPACK.

27. PYTORCH FOUNDATION. *Core ML Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/coreml/coreml-overview.html.

28. APPLE. *Core ML* [online]. 2026. [visited on 2026-04-14]. Available from: https://developer.apple.com/documentation/coreml.

29. PYTORCH FOUNDATION. *Quantization Overview — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/quantization-overview.html.

30. PYTORCH AO CONTRIBUTORS. *PT2E Quantization Tutorials and Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/ao/main/tutorials_source/pt2e_quant_ptq.html.

Contents of the attachment

The electronic attachment contains the source code of the benchmark applications, benchmarking scripts, collected run exports and the thesis sources.

```
/
├── android/ ........................ Android benchmark app source code
├── ios/ ............................... iOS benchmark app source code
├── benchmarks/ ......... Benchmark harness, analysis scripts, and results
│   ├── analysis/ .... JSON aggregation, charting and table export scripts
│   └── results/ ....... Collected exported runs + generated charts/tables
├── models/ .......... Detector and landmark model files used by the apps
└── thesis/ ................................ LaTeX sources of the thesis
    ├── ctufit-thesis.tex ...................... Main thesis entry point
    └── text/ ........................................... Chapter sources
```